Optimizer review, fixes, tests and refactor

Lambach Experimental Aircraft · version_1 · branch **optimization** (created from **dev**) · 12 Nov 2026

Summary

This report covers the work on `version_1/src/Dev/optimizer.py`, the AeroSandbox/IPOPT optimizer that finds the minimum-drag, trimmed, statically stable design of the four-prop body-lift aircraft at 200 km/h. The work was done in this order: review the logic, fix the mistakes, run the code, write pytests, refactor into the README layout using `global_parameters`, and check that the refactored code behaves the same as the original.

- **The original code could not produce a result.** IPOPT stopped at iteration 0 with *Invalid_Number_Detected*: the fuselage nose and tail radius were a symbolic √0, whose derivative is infinite.
- **Once that was fixed, the result was still wrong.** The propeller table was cut off at J = 0.86, so the four props could not make less than about 2.8 N of thrust at 200 km/h. Since thrust has to equal drag, the optimizer made the aircraft *draggier* (0.55 m fuselage, 8° AOA) instead of less draggy, and never converged. The fix extends each RPM block of the APC data the last short step to zero thrust.
- **After the fixes:** the baseline converges in 11 iterations (0.9 s) to a drag of **1.308 N** (previously stuck at 2.74 N and infeasible). All 8 configuration variants tested now converge.
- **Tests:** 70 pytests, all passing. Run against the original unfixed code, the suite catches both bugs (12 tests fail or error).
- **Refactor:** 6 helpers and 7 modules under `src/optimization/`, one class or function per file. The `Assumptions` class is now filled in (before, the file did not even import), and g, ρ and a come from `Parameters`. `main.py` wires everything together.
- **Equivalence:** with the old physical constants, the refactored code matches the original to 1e-5 in all 5 reference cases. The propeller tables and AeroBuildup outputs are bit-for-bit identical. With the new G0 = 9.80665 (weight −0.035 %), every dimensional result moves by less than 0.07 %.

1. Branch and environment

Following the README workflow (one branch per `src` subfolder, created from dev, never committing to dev or `main`), a new branch **optimization** was created from dev (a7a3d2d). No `.venv` existed, so one was created exactly as the README describes (`pip install -r requirements.txt`, plus pytest). It is gitignored. Versions: Python 3.12.14, aerosandbox 4.2.10, casadi 3.8.1, numpy 2.5.3, pytest 9.2.0.

Before the refactor, four decisions were checked with you:

Question	Your decision
How to remove the propeller thrust floor	Extrapolate each RPM block linearly to zero thrust
Subfolder / branch name	`optimization`; data files moved to the top-level `data/`
Use of global_parameters	Fill `Assumptions` with Config values; use `Parameters` (G0) for physics
The old optimizer.py	Move it to `src/deprecated/` and remove `src/Dev/`

2. Logic review

The whole file was read, and every sign convention that the physics relies on was checked numerically against AeroSandbox. The results fall into three groups: bugs (fixed), things that are correct, and modelling observations (reported here, not changed, because they are design choices).

2.1 Mistakes found and fixed

#	Problem	Effect	Fix
1	`build_fuselage` computed r = b·√max(1 − (x/a)², 0) with x = −a·cos β. At the nose and tail this is a symbolic √0, and d√u/du → ∞; ∞·0 = NaN in the gradient with respect to `fuse_len`.	**Solver aborts at iteration 0** (*Invalid_Number_Detected*). The code never produced a result.	r = b·sin β. This is exactly the same radius, but its derivative is finite everywhere.
2	The J grid was capped at the *shortest* RPM block (35 000 RPM stops at J = 0.864, rounded down to 0.86). Every other block runs on to J ≈ 0.882–0.885, where Ct ≈ 0.	**Artificial thrust floor** of ≈ 2.8 N at 200 km/h. Because thrust = drag, the optimizer inflated drag to 2.74 N, the fuselage went to its 0.6 m limit, and the problem did not converge in 1000 iterations.	Extend each block with a straight line through its last 3 points out to Ct = 0 (a step of ≤ 0.02 in J). The table limit becomes J = 0.881.
3	`global_parameters.py`: `Assumptions.__init__` had only comments in its body.	**IndentationError**: the file could not be imported at all.	Filled in during the refactor (section 5).
4	The report's "L/D" included propeller lift.	Overstated the airframe L/D.	The report now prints the aerodynamic L/D and the L/D including props separately.
5	A wrong comment about the wind-axis +x direction, and a misindented comment.	Cosmetic.	Corrected.

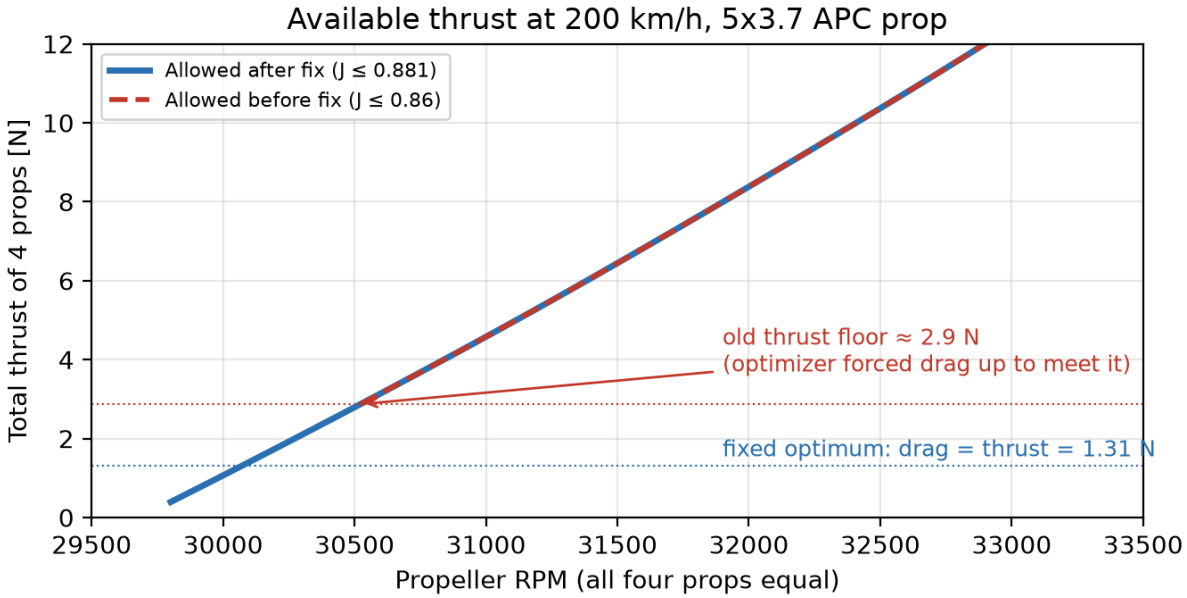


Figure 1. Total thrust available from the four props at 200 km/h. With the old J limit (red), the thrust could not drop below ≈ 2.9 N (≈ 2.8 N with the two pairs at different RPM). After the fix (blue), the props can match the true minimum drag of 1.31 N.

2.2 Verified correct

- **Axes.** AeroSandbox geometry axes are +x aft and +z up. Checked at α = 10°: forward thrust maps to +0.985 T along wind-x (which opposes D = $-F_{w,x}$) and gives +0.17 T of lift (L = $-F_{w,z}$). A geometry-axis M_y equals the body-axis M_y (nose-up positive), which is the same sign as AeroBuildup's m_b.
- **Thrust moment.** The moment −z·ΔT·cos i − Δx·T·sin i matches an independent r × F sum over the four props (covered by a test). The bottom pair pushing harder gives nose-up.
- **Neutral point.** AeroBuildup's x_{np} is about the fuselage centre; converting it to nose-based (+L/2) is correct.
- **APC columns and propeller formulas.** The columns are read correctly (J = 1, Ct = 3, Cp = 4). T = Ct·ρn²D⁴, P = Cp·ρn³D⁵, Q = P/(2πn), and the tip Mach includes the axial speed.

- **Constraints.** The prop-spacing constraint ($l_t \geq D + 1$ cm), the fuselage-diameter-at-station constraint, and the layout and CG constraints are all geometrically consistent.

2.3 Modelling observations (not changed — design decisions for the team)

- **The prop is mismatched to 200 km/h.** At the optimum, J ≈ 0.87–0.88 is almost windmilling, and the propulsive efficiency is 0.156 (465 W of shaft power to deliver 73 W of useful power). A higher-pitch prop, or minimizing power instead of drag, would change the design a lot.
- **The design is mostly decided by bounds.** CG = 50 mm (its lower bound) forces l_p = 200 mm through `fix_l_cg`. Canard chord, pylon thickness and l_c are at their minimums, ΔT is at its ±50 % limit, and the static margin is exactly 2 %. There is no mass model, so the CG can be placed anywhere.
- **Reference area and chord come from the first wing** (`pylons_upper`). CLa and Cma are therefore scaled by one pylon pair; the dimensional forces and x_{np} are unaffected.
- **The local stall guards are geometric** (AOA + incidence): no canard downwash and no propwash. The neutral point ignores the propellers' normal force.
- **Minor:** with `fix_l_cg=False`, the constraint $l_{cg} \leq x_{np}$ is already implied by the static-margin constraint. The `climb_allowed` case (steady_level=False) lets thrust exceed drag, i.e. the aircraft accelerates.

3. Running the code

Runs before and after the fixes. "NaN fix only" means fix 1 alone, to show what fix 2 changed on its own.

Configuration	Original	NaN fix only (1000 iter max)	Both fixes
baseline	**FAIL** iter 0, NaN	**FAIL** infeasible, D = 2.739 N	**OK** 11 iter, D = 1.308 N
fix_l_cg = False	**FAIL** NaN	**OK** D = 2.758 N	**OK** D = 1.306 N
fix_pylon_len = False	**FAIL** NaN	**OK** D = 2.832 N	**OK** D = 1.328 N
fix_fuse_dia = False	**FAIL** NaN	**OK** D = 2.757 N	**OK** D = 1.276 N
all three free	**FAIL** NaN	**FAIL** AOA at 12° bound, D = 2.785 N	**OK** D = 1.278 N
use_diff_thrust = False	**FAIL** NaN	**FAIL** D = 2.725 N	**OK** D = 1.309 N
use_canards = False	**FAIL** NaN	—	**OK** D = 2.120 N
steady_level = False	**FAIL** NaN	**OK** D = 1.117 N	**OK** D = 1.117 N

Every steady-level run with only the NaN fix finished at J = 0.860 exactly, the old table edge, which is how the thrust floor showed up. Only the inequality case (steady_level=False) could escape it.

3.1 Final baseline result — is it reasonable?

Quantity	Value	Quantity	Value
Drag (profile + induced)	1.308 N (1.071 + 0.238)	Fuselage L × D	286.2 × 73.4 mm
Lift aero + props / weight	8.267 + 0.071 / 8.338 N	Canard span × chord, inc.	114.0 × 60.0 mm, −0.60°
Aerodynamic L/D	6.32	Pylon length × chord, inc.	65.0 × 75.0 mm, −0.13°
AOA	3.25°	CG / neutral point (from nose)	50.0 / 55.7 mm (SM 2.0 %)
RPM upper / lower	29 835 / 30 219	Thrust per prop up / low	0.164 / 0.491 N
ΔT (upper − lower)	−0.655 N (at the 50 % limit)	Trim: thrust vs aero moment	+0.0470 vs −0.0470 N·m
Shaft power	464.5 W (≤ 126 W per motor)	Tip Mach	0.605 / 0.613
Propulsive efficiency	0.156	J (table limit 0.881)	0.878 / 0.867

Verdict. The result is physically consistent: lift = weight, thrust = drag, and the trim moment is ≈ 1e-15 N·m. Pitch trim is achieved with differential thrust, exactly as the stability README intends (the lower pair works harder to cancel a nose-down aerodynamic moment). An L/D of about 6 is believable for a body-lift vehicle with short surfaces. The two red flags are the observations in 2.3, not code errors: the props are nearly windmilling (η ≈ 0.16), and most of the geometry sits on its bounds.

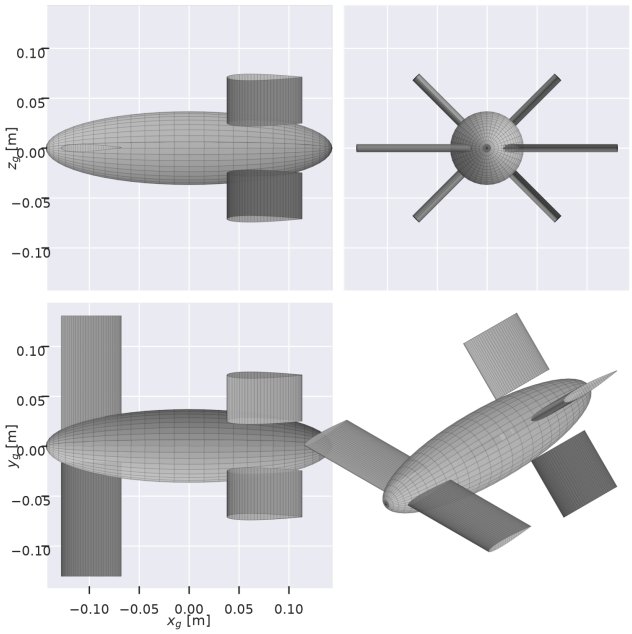


Figure 2. Three-view of the optimized baseline design (saved by main.py to version_1/figures/).

4. Tests

All tests live in `version_1/tests/optimization/` (flat, mirroring `src/optimization` as the README requires). First, characterisation tests were written for the fixed original code, together with a golden reference file. This file stores every design variable and 26 outputs for 5 configurations, generated by `make_reference_results.py` from the original code. It is the record of the "previous code's" results that the refactor has to reproduce.

File	Tests	What it covers
test_legacy_optimizer.py	23	Original code: helpers, propeller table (range, zero-thrust extension, agreement with the raw file, T/P/Q/J relations, bad file), geometry (component count, ellipsoid, X-pylons on the fuselage surface), AeroBuildup sanity, solved invariants (force/moment balance, every constraint, hand-computed r × F moment, trim direction), build_solved_airplane, golden regression ×5
test_optimization_helpers.py	18	All 6 helpers, including the axis sign checks over a 3 × 3 grid of α and incidence
test_optimization_modules.py	16	Assumptions, the propeller database, PropulsionSystem (with and without diff thrust, bad RPM range), geometry, AeroBuildup, solved invariants, the report
test_refactor_equivalence.py	13	Old vs new (section 6)
Total	**70**	**70 passed** in 89 s

Do the tests catch the bugs? The same suite was pointed at the original unfixed file from dev (using the `LEGACY_OPTIMIZER` environment variable): **7 failed, 5 errors**, 11 passed. All solver tests fail with *Invalid_Number_Detected*, and the table tests fail on the 0.86 cap. A tiny edge case found while writing the tests was also fixed: a block ending at exactly Ct = 0 got a duplicate J point. This changes no real result (the reference file regenerated byte-identical).

5. Refactor

The code was split following the README rules: lowercase names, exactly one class or function per file, small reusable functions in `helpers/`, phases in `modules/`, and the assembly only in `main.py`.

```
data/                                  PER3_5x37E-3_performance_file.txt, 5x37E-3-PERF_geometry.txt
version_1/
├── main.py                            assumptions → prop database → optimizer → report → figure
├── figures/optimized_aircraft_three_view.png
├── src/global_parameters/global_parameters.py   Parameters (+ speed_of_sound_sea_level), Assumptions
├── src/optimization/helpers/
│   ├── radius_from_ellipse.py         polar_to_cartesian.py
│   ├── read_apc_performance_blocks.py extend_to_zero_thrust.py
│   └── thrust_force_components.py     thrust_pitching_moment.py
├── src/optimization/modules/
│   ├── propeller_performance_database.py   PropellerPerformanceDatabase
│   ├── propulsion_system.py                PropulsionSystem
│   ├── design_variables.py                 DesignVariables (new: was inside the optimizer)
│   ├── aircraft_aerodynamics.py            AircraftAerodynamics
│   ├── apply_design_constraints.py         apply_design_constraints()
│   ├── aircraft_design_optimizer.py        AircraftDesignOptimizer
│   └── print_design_report.py              print_design_report()
├── src/deprecated/optimizer.py        original file (fixed), data path updated
└── tests/optimization/                conftest, reference data, 4 test files
```

5.1 How global_parameters is used

Before (in optimizer.py)	After
`Config` class (switches, flight condition, prop limits, CG offset, file path)	`Assumptions`, grouped under its existing headings (switches, flight conditions, structural, engine, mass, fuselage)
Hard-coded numbers in the code: 0.07, 0.10, 0.16, 0.12, 0.065, 0.0635 + 0.01, 2·0.0635 + 0.01, 0.012, 0.03, 0.02	Named fields: fuse_dia_min, l_min_dia, tc_pylon, tc_canard, forced_pylon_len, prop_radius + prop_clearance_gap, prop_dia + prop_spacing_gap, pylon_thick_min, tail_margin, canard_cg_gap
weight = 0.85 × 9.81	aircraft_mass × `Parameters.G0` (9.80665), as a property
ρ, a from `asb.Atmosphere(altitude=0)`	`Parameters.AIR_DENSITY_SEA_LEVEL`; a = √(γRT) from Parameters

Variable bounds and initial guesses stay with the variables in `DesignVariables` and `PropulsionSystem`, because they are solver settings, not physical assumptions. The order of variables and constraints was kept identical, so IPOPT follows the same path. Small behaviour changes: `solve()` no longer prints the zero-tolerance infeasibility list after a successful solve (it was all ~1e-8 noise), it accepts `verbose`, and `main.py` saves the three-view to `figures/`. `global_parameters.py` still holds two classes; that was already the case before, so it was left as it is.

6. Old vs new comparison

Check	Tolerance	Result
Propeller tables (rpms, J grid, Ct, Cp) and evaluate() at 3 points, legacy vs new	bit-exact	**PASS**
AeroBuildup L, D, D_profile, D_induced, m_b, CLa, Cma, x_np for the same design, with and without canards	bit-exact	**PASS**
Full optimization, 5 cases, new code with the **legacy** constants (g = 9.81, ρ and a from AeroSandbox's ISA) vs the golden reference: 38 quantities per case, plus weight and J limit	rel 1e-5	**PASS 5/5**
Full optimization, 5 cases, new code with the **new** global Parameters vs the golden reference	rel 1e-3, abs 5e-5	**PASS 5/5**

The new constants change the weight by −0.035 % (and ρ by 7e-7, a by 5e-6). The largest resulting differences across all 5 cases:

Quantity	Max rel. diff.	Max abs. diff.	Comment
drag_induced	6.7e-4	5.7e-4 N	largest change in any dimensional output
lift_aero / lift_tot	5.2e-4 / 3.4e-4	2.1e-3 / 2.9e-3 N	follows the weight
total_drag	3.2e-4	6.8e-4 N	
AOA	4.8e-4	2.7e-5 rad	0.0016°
pylon_inc, canard_inc	2.4e-3, 1.5e-3	≤ 1.5e-5 rad	near-zero values (≈ 0.0005 rad), hence the absolute tolerance

Quantity	Max rel. diff.	Max abs. diff.	Comment
rpm_top / power_tot	1.0e-5 / 6.6e-5	0.38 RPM / 0.033 W	
geometry (l_cg, l_c, chords, pylon_len)	< 1e-7	< 3e-9 m	set by active bounds

Conclusion: the refactored code solves the same problem and returns the same answer. All remaining differences are fully explained by the deliberate switch to the shared G0 constant.

7. Commits and how to run

Commit	Content
1f03782	Fix the NaN gradient and the artificial propeller thrust floor (in src/Dev/optimizer.py)
68948af	Characterisation tests and golden reference for the legacy optimizer
d5d5ea3	Refactor into src/optimization with global parameters; data/ and deprecated/ moves; equivalence tests

```
source .venv/bin/activate
python version_1/main.py                                        # optimize, print report, save figure
python -m pytest version_1/tests/optimization                   # 70 tests, ~90 s
python version_1/tests/optimization/make_reference_results.py   # only after an intended physics change
```

Nothing has been pushed, and nothing was committed to dev or `main`. The `optimization` branch exists only locally.

8. Suggested next steps

- Choose a propeller suited to 200 km/h, or add a second PER3 file and compare. η ≈ 0.16 is where most of the power is lost.
- Consider minimizing shaft power (or adding it to the objective) instead of drag only.
- Add a simple mass/CG model, so that l_{cg} is not a free variable pinned at its lower bound.
- Set s_{ref} and c_{ref} on the asb.Airplane explicitly (e.g. body frontal area and length, as discussed in the stability README), so CLa and Cma are meaningful.